

ParkVision 系统实现答辩讲稿（15 分钟版）

建议语速：正常偏快。33 页不能平均讲，每页只讲一个重点。截图页快速带过，架构、核心代码、后端、AI 重点展开。

时间分配

部分	页码	时间
开场与目录	1-2	40 秒
项目回顾	3-4	1 分 20 秒
项目工程结构	新增代码架构页	40 秒
小程序端实现	5-11	3 分钟
控制台端实现	12-20	3 分钟
后端微服务实现	21-27	4 分钟
AI 功能实现	28-32	2 分 30 秒
项目总结与致谢	33-34	40 秒

逐页讲稿

第 1 页 封面

各位老师好，我们小组本次汇报的题目是 ParkVision，基于边缘计算与多智能体协同的物理信息智慧停车系统。本次汇报重点不是开题阶段的需求设想，而是我们在系统实现阶段完成的前端页面、后端架构和 AI 功能接入设计。下面由我来介绍。

第 2 页 目录

这次汇报分为五个部分。首先快速回顾项目整体目标，然后介绍车主小程序端、运营控制台端，接着重点说明后端微服务实现，最后介绍 AI 功能如何接入停车业务闭环。

第 3 页 项目回顾

ParkVision 面向的是立体智慧停车场景。传统停车系统主要解决“记录车位”和“收费”的问题，而我们的项目希望进一步解决三个问题：第一，车主端使用是否方便；第二，管理端能否实时掌握车库运行状态；第三，AI 感知结果能否真正参与后端业务决策。

第 4 页 逻辑架构图

从整体架构看，系统分为四层：用户交互层、业务服务层、AI 感知层和数据支撑层。车主通过小程序完成入场、取车、临时取物等操作；管理人员通过控制台查看车位、AGV、告警和报表；后端负责统一业务接口；AI 模块负责把

摄像头画面转化为结构化识别结果。我们实现时重点围绕这条“前端操作、后端处理、AI 辅助决策”的闭环展开。

新增代码架构页 项目工程结构

这一页展示的是我们项目的整体工程目录。ParkVision-System 采用前后端分离的工程组织方式，整体目录按照业务开发、部署运维、文档沉淀和启动脚本进行划分，便于小组协作和后续扩展。

其中 backend 是 Spring Boot 后端服务，负责业务接口、订单状态、车位管理、调度与 AI 结果接入；frontend 是 Vue 3 前端工程，包含车主端展示、运营控制台、AI 感知页面等核心界面；deploy 和 scripts 用于部署配置与一键启动停止；docs 用于保存架构图、接口文档和答辩材料；logs 用于记录运行日志，方便调试接口和排查问题。

这页可以强调一句：相比最开始比较零散的文件结构，我们后面把项目整理成了标准的前后端分离工程，后端、前端、文档、部署和脚本都有独立目录，因此项目不只是页面 Demo，而是具备继续开发、部署和维护的完整工程基础。

第 5 页 小程序端章节页

下面进入小程序端实现。小程序端主要面向车主，因此我们的设计目标是流程清晰、操作少、状态反馈明确。

第 6 页 小程序代码架构

小程序端采用 uni-app、Vue 3 和 TypeScript。这样做的好处是既能贴近微信小程序生态，又能使用 Vue 3 的组合式 API 提高代码复用性。工程结构上，我们把请求封装放在 api 模块，把通用 UI 放在 components 模块，把停车、车辆、用户等核心流程放在 pages 模块。TypeScript 主要用于约束接口返回结构，减少前后端字段不一致带来的问题。

第 7 页 我的车辆页面：视觉层

这里以“我的车辆”页面为例。这个页面不是单纯展示车辆信息，而是承载车主最核心的操作入口。我们用不同颜色区分车辆状态，例如存放中、取物中、取车中等。这里还加入了两个项目特色功能：临时取物和 VIP 加急取车。它们在界面上是按钮，在业务上对应的是不同优先级的调度请求。

第 8 页 我的车辆页面：逻辑层

这一页展示的是页面背后的逻辑。我们使用 script setup 组织状态和方法，通过 handleAction 统一处理取车、临时取物、加急取车等操作。因为停车调度涉及 AGV 资源，不能让用户连续点击造成重复任务，所以前端使用 loading 和异步等待机制进行防抖。这样页面操作和后端调度之间就形成了比较稳定的交互流程。

第 9 页 C 端登录页面

登录页面主要解决身份入口问题。这里我们重点保证页面简洁，用户可以快速进入系统。答辩中这页可以快速带过，说明我们完成了 C 端基础登录流程和统一视觉风格。

第 10 页 C 端个人主页

个人主页负责聚合用户信息、车辆信息和停车记录入口。它的作用是把分散的停车操作集中到车主视角下，方便用户查看自己的停车状态和历史信息。

第 11 页 AI 助手页面

这一页是 C 端对 AI 能力的展示。AI 助手不是为了替代按钮操作，而是提供自然语言入口。比如用户可以询问车辆状态、取车预计时间，系统后续可以把自然语言解析成查询订单、查询车位或发起调度等业务动作。

第 12 页 控制台端章节页

接下来是控制台端。控制台主要面向停车场运营人员，所以重点不是美观展示，而是实时监控、调度管理和异常处理。

第 13 页 控制台代码架构

控制台采用 Vue 3、Vite、Element Plus 和 ECharts。Vue 3 负责组件化开发，Vite 提升开发效率，Element Plus 提供后台管理组件，ECharts 用于展示车位统计、车流趋势和运营报表。状态管理上使用 Pinia 维护车位、AGV、告警等全局状态，便于多个页面共享数据。

第 14 页 数字孪生监控大屏：页面实现

数字孪生大屏是控制台的核心页面。我们把物理车库抽象成可视化网格，用不同颜色表示空闲、占用、故障等状态。页面中还包含实时告警和底层数据表格，方便运营人员既能看整体，也能追踪具体车位。

第 15 页 数字孪生监控大屏：核心逻辑

这一页是数字孪生的技术重点。后端返回的是离散的车位对象和 AGV 坐标，前端需要把这些数据映射成标准网格。我们通过 computed 计算属性完成状态转换，把物理坐标转成网格索引，同时判断每个格子是否有车辆、是否有 AGV、是否处于异常状态。这样后端数据变化后，前端视图可以自动刷新。

第 16 页 总览页面

总览页面用于给运营人员提供全局状态，包括当前车位使用情况、今日订单、收入和异常情况。这一页主要体现控制台的信息汇总能力。

第 17 页 AI 感知页面

AI 感知页面展示摄像头识别结果，包括车辆、车牌和异常事件。它对应后面 AI 模块的前端展示层，让管理人员知道 AI 当前识别到了什么，以及识别结果是否需要人工复核。

第 18 页 调度中枢页面

调度中枢页面用于管理 AGV 调度任务，例如入库、出库、临时取物和加急取车。这里体现的是系统从“记录停车”进一步走向“调度停车”的能力。

第 19 页 报表与记录页面

报表页面主要服务运营分析，包括订单记录、收入趋势和车位使用情况。它为后续优化车位分配、调度策略和高峰预测提供数据基础。

第 20 页 网关与配置页面

网关与配置页面用于展示系统参数、接口配置和设备接入信息。这部分说明我们的控制台不仅能看业务，也预留了和边缘设备、AI 服务、后端接口对接的管理入口。

第 21 页 后端微服务章节页

下面是本次实现中比较重要的一部分：后端微服务实现。我们目前完成的是面向微服务拆分的后端分层架构，重点是接口清晰、业务模块清晰，并且演示阶段具备稳定的数据回退能力。

第 22 页 后端代码架构

后端采用 Spring Boot 3 和 Java 17。代码按照 controller、service、repository、domain、dto、common、config 分层。Controller 负责对外暴露 RESTful API，Service 处理业务流程，Repository 抽象数据访问，Domain 表示核心业务对象，DTO 用于前后端传输。这样划分之后，前端、AI 和数据源之间不会直接耦合，后续接入 MySQL、Redis 或消息队列时，只需要替换底层实现。

第 23 页 Controller 与 Service

这一页以订单模块为例说明后端分层。OrderController 负责接收前端请求，比如模拟入场、取车、临时取物和支付完成；OrderService 负责真正的业务状态流转，比如修改订单状态、读取车位数据、保存订单结果。也就是说，Controller 不直接操作数据，而是把请求交给 Service，Service 再通过 ParkVisionRepository 获取或保存业务数据，最后统一返回 ApiResponse。

第 24 页 数据设计：核心业务关系图

数据设计围绕停车业务主线展开。车主拥有车辆，车辆产生停车订单，订单关联具体车位。AGV 调度任务记录车辆搬运动作，AI 告警事件记录摄像头识别出的异常情况。这里的核心是停车订单，因为它连接了用户、车辆、车位、计费 and 调度，是整个系统业务闭环的主线。

第 25 页 库位状态模型

库位模型 ParkingSlot 描述物理车位，包括车位编号、所在层级和当前状态。我们把状态设计成枚举，例如 EMPTY、OCCUPIED、CHARGING、BUFFER、MAINTENANCE。这样做比直接使用字符串更稳定，也方便前端根据状态渲染不同颜色，后端根据状态判断是否可以分配车位。

第 26 页 订单状态机

订单状态机用于控制停车业务生命周期。车辆入库后订单进入 PARKED 状态；用户取车时进入 RETRIEVING；临时取物时进入 TOUCHING；支付完成后进入 FINISHED。通过状态机限制流程，可以避免订单被随意修改，也能让计费、调度和页面显示都围绕同一套状态规则运行。

第 27 页 Repository 抽象与 fallback 数据源

这一页是后端演示稳定性的关键。Service 层只依赖 ParkVisionRepository 接口，不关心数据到底来自数据库还是内存数据。当前答辩阶段，我们使用 FallbackParkVisionRepository 提供稳定演示数据，保证即使数据库或外部服务没有完全接入，前端页面和后端接口也能正常展示。后续要接入 MySQL、Redis 或 RabbitMQ，只需要新增 Repository 实现，不需要大改业务层。

第 28 页 AI 功能章节页

最后介绍 AI 功能实现。我们这部分强调的不是单独训练一个模型，而是 AI 识别结果如何进入系统业务流程。

第 29 页 AI 功能概述

ParkVision 的 AI 能力分为三类。第一类是视觉识别，包括车辆检测、车牌识别和人员闯入检测；第二类是预测调度，根据车流高峰提前优化车位和 AGV 任务；第三类是智能助手，把自然语言请求转成查询或调度动作。这三类能力分别对应感知、预测和交互。

第 30 页 模型选取与推理流程

在模型选取上，我们设计使用 YOLOv8 进行车辆、车位和人员区域检测，使用 HyperLPR 或 OCR 识别车牌号码，并通过 ROI 区域判断是否存在人员闯入或异常停放。当前阶段为了保证系统联调稳定，使用模拟推理结果，但接口字段和真实模型服务保持一致。这样后续替换成真实边缘推理服务时，前后端业务代码不需要大改。

第 31 页 AI 与后端对接

AI 模块通过统一接口进入 Java 后端，也就是 VisionController。推理结果会被封装成 JSON，交给 VisionService 判断 action 字段。如果 action 是放行入场，就可以触发订单创建和车位更新；如果是异常事件，就触发告警；如果涉及取车高峰，也可以联动调度模块。这里的重点是 AI 和业务模块通过 JSON 解耦，AI 只负责提供结构化判断，业务规则仍然由后端控制。

第 32 页 AI 返回结构

这一页展示 AI 返回结果的核心字段。plate 表示识别到的车牌号，confidence 表示置信度，intrusion 表示是否存在人员闯入，action 表示建议执行的业务动作。通过这个结构，AI 结果就不只是页面上的展示信息，而是可以直接驱动后端订单、车位、告警和调度流程。

第 33 页 项目总结

这一页建议用四个小模块总结，不要写长段落。标题可以写“项目实现总结”。

PPT 页面文字建议如下：

项目实现总结

目前 ParkVision 已完成从前端交互、后端接口到 AI 感知接入的系统实现雏形，具备完整演示链路。

- **前端页面实现**：完成车主端登录、个人主页、车辆管理、AI 助手，以及控制台总览、数字孪生、AI 感知、调度中枢、报表记录等页面。
- **后端服务实现**：完成 Spring Boot 分层架构，形成 Controller、Service、Repository、Domain、DTO 的标准工程结构，并提供统一 RESTful API。
- **核心业务闭环**：实现车位状态管理、停车订单流转、临时取物、VIP 加急取车、AGV 调度任务和异常告警等核心业务流程。
- **AI 功能接入**：设计 YOLOv8、HyperLPR、ROI 安全检测推理链路，通过 JSON 结果接入后端，联动车位、订单、告警和调度模块。

这页讲的时候可以这样说：

总结一下目前的实现成果。前端方面，我们已经完成了车主端和运营控制台的主要页面，能够覆盖用户停车、管理监控和调度查看等场景。后端方面，我们完成了基于 Spring Boot 的标准分层结构，并通过统一接口支撑前端页面和 AI 模块。业务方面，系统已经形成车位状态、停车订单、AGV 调度和异常告警的基本闭环。AI 方面，我们设计了从摄像头识别到 JSON 推理结果，再进入后端业务系统的接入流程。虽然当前阶段部分数据仍然使用 fallback 方式保证演示稳定，但整体架构已经具备继续接入真实数据库、真实模型服务和部署环境的扩展基础。

第 34 页 结束页

以上就是我们 ParkVision 系统实现部分的汇报。总结来说，我们完成了车主端、运营控制台端、后端分层架构和 AI 接入流程，形成了从用户操作、车库管理、AI 感知到后端业务处理的完整闭环。我的汇报到此结束，请各位老师批评指正。

如果时间只剩 12 分钟，压缩方法

第 9、10、16、19、20 页只用一句话带过；第 7 和第 8 页合并讲；第 14 和第 15 页合并讲；后端第 22 到第 27 页不要删，这是拿分重点；AI 第 30 到第 32 页也不要删，这是体现创新性的重点；第 33 页项目总结保留，但控制在 40 秒以内。